Universidad Tecnológica de Bolivar
Ingeniería de Software Código 1206

Parcial práctico de Ingeniería de Software

La última transmisión

Motor de historias interactivas: requisitos, modelado, arquitectura, implementación y cambio

Integrantes

Zulianys Liseth Orozco Chavez	T00078330
Juan David Hernandez Avila	T00081559

Cartagena, octubre de 2026

1. Fundamentos: problema, alcance y calidad

1.1. Problema

Un colectivo de escritores prepara una ficción interactiva para un festival universitario. Sus integrantes no programan, pero necesitan crear y modificar escenas y decisiones sin pedir cambios en el programa. Sin una herramienta que interprete sus narraciones, cada ajuste de contenido depende de un desarrollador y un error en el relato puede llegar al lector como un cierre inesperado. El proyecto desarrolla un motor que carga narraciones desde archivos, ofrece solo las decisiones compatibles con lo ocurrido en la partida, conserva ese estado y valida el contenido antes de que alguien lo juegue.

1.2. Actores

Actor	Interés principal	Necesidades
Escritora principal	Editar escenas y decisiones en un archivo, sin tocar el código	N01, N06
Diseñador de la experiencia	Decisiones con consecuencias y opciones que dependen de lo descubierto	N02, N03, N04
Coordinadora del festival	Saber qué corregir en una historia con errores; que el lector no sufra cierres	N06
Lector	Leer, elegir, llegar a un desenlace y comenzar otra partida	N02, N03, N05
Equipo de desarrollo	Entregar motor, pruebas y evidencia verificables	N01–N06

1.3. Alcance

Incluye: carga de historias desde archivos independientes; estado inicial declarado en el archivo; condiciones de igualdad sobre variables booleanas y efectos que asignan verdadero o falso; estado de la partida en memoria; reconocimiento de finales y reinicio; validación previa del contenido (nueve categorías de error); interfaz [de consola: ajustar si es otra]; ejecución local; la historia de referencia de seis escenas y una segunda historia.

Excluye: cuentas, base de datos, guardado entre ejecuciones, conexión a internet, editor visual, audio, imágenes, lenguaje de expresiones y análisis automático de todos los caminos del relato.

Decisión pendiente del cliente: la escritora quiere permitir retroceder siempre y el diseñador quiere decisiones irreversibles. La versión inicial no implementa retroceso; la pregunta de aclaración y el acuerdo provisional se registran en la sección de requisitos, sin presuponer la decisión del cliente.

1.4. Riesgos

ID	Riesgo	Prob./Imp.	Mitigación y verificación
R1	Contenido inválido escrito por personas no programadoras provoca cierres o bloqueos durante la partida.	Alta / Alto	Validación previa de las nueve categorías de error con mensaje de causa y campo o ID, y manejo controlado de entradas inválidas. Se verifica con pruebas parametrizadas (RNF02, RNF03).
R2	El motor queda acoplado a la historia de referencia (identificadores de escena en el código) y la escritora vuelve a depender de programadores.	Media / Alto	El motor evalúa condiciones y destinos solo desde los datos; revisión del código en busca de identificadores de escena y prueba con una segunda historia sin editar el motor (RNF01).

1.5. Priorización de atributos de calidad

1. **Modificabilidad del contenido (RNF01).** Es la razón de ser del proyecto: la escritora pide agregar escenas y cambiar decisiones sin modificar el programa.
2. **Claridad de errores (RNF02).** Un escritor no programador solo puede aprovechar esa libertad si sabe qué corregir; la coordinadora lo pide de forma explícita.
3. **Confiabilidad (RNF03).** El lector no debe encontrarse con un cierre inesperado. Se atiende en buena medida con la misma validación previa de los dos atributos anteriores.

No se priorizan el rendimiento, la seguridad ni la portabilidad avanzada: las historias son pequeñas, se ejecutan localmente y no hay usuarios con datos sensibles. El costo aceptado es que la validación previa añade trabajo al cargar, despreciable a esta escala.

1.6. Decisión de responsabilidad profesional: autoría y dependencias

Hay trazabilidad de autoría, instalación reproducible y licencias conocidas. El motor no almacena ni transmite datos personales, porque el estado vive solo en memoria durante la partida. El costo es la disciplina de llevar el registro mientras se trabaja.

2. Requerimientos

2.1. Requerimientos funcionales

ID	Requisito	Criterio de aceptación
N01. Los escritores necesitan distribuir relatos en archivos independientes y definir dónde y con qué información comienza cada partida.		
RF1.1	**Carga de historia externa.** El sistema debe permitir cargar los datos de la historia desde un archivo externo independiente del código fuente.	Al especificar la ruta de un archivo válido, el motor lee su contenido en memoria de forma exitosa sin modificar el código fuente del programa.
RF1.2	**Inicialización del estado.** El sistema debe establecer la escena inicial y los valores por defecto de las variables booleanas a partir de la información declarada en el archivo.	Al iniciar una partida nueva, el sistema sitúa al jugador en el identificador indicado en el campo «inicio» y carga el mapa de variables en su estado original. Dos archivos distintos cargan dos historias distintas.

ID	Requisito	Criterio de aceptación
N02. Los lectores necesitan leer la escena actual y ver únicamente decisiones compatibles con la información obtenida.		
RF2.1	**Presentación de la escena.** El sistema debe mostrar en pantalla el texto descriptivo correspondiente a la escena activa.	Al cambiar de escena, se imprime el contenido exacto del campo «texto» de la nueva escena.
RF2.2	**Filtrado de opciones por condiciones.** El sistema debe evaluar el mapa de condiciones de cada opción contra el estado actual de las variables y mostrar únicamente las opciones habilitadas.	Si una opción requiere que una variable sea verdadera y en el estado actual es falsa, la opción no se imprime en pantalla. Un mapa de condiciones vacío habilita la opción automáticamente.
N03. Elegir debe producir las consecuencias indicadas en el relato y llevar al destino previsto. Los errores de entrada no deben alterar la partida.		
RF3.1	**Transición de escenas.** Al recibir una elección válida, el sistema debe cambiar la escena activa al identificador indicado en el campo «destino» de dicha opción.	Si la opción seleccionada tiene como destino «E4», la siguiente pantalla muestra el contenido de la escena «E4».
RF3.2	**Inmunidad ante errores de entrada.** El sistema debe rechazar cualquier entrada del usuario que no corresponda a una opción actualmente habilitada.	Si el usuario ingresa un comando inválido o selecciona una opción inhabilitada o inexistente, el sistema lo rechaza en el motor, vuelve a pedir la entrada y el estado de la partida queda intacto.
N04. La experiencia debe recordar durante la partida qué ha ocurrido para que las elecciones posteriores tengan consecuencias coherentes.		
RF4.1	**Aplicación de efectos.** Tras una selección válida, el sistema debe actualizar el estado en memoria aplicando las asignaciones indicadas en el mapa de «efectos» de la opción elegida.	Si la opción elegida contiene el efecto «`conoce_origen` = verdadero», la variable `conoce_origen` cambia a verdadero en la memoria de la sesión actual y permanece así hasta que la partida termine.
N05. El lector necesita reconocer el desenlace y comenzar otra partida sin conservar información de la anterior.		
RF5.1	**Detección de final.** El sistema debe detectar cuándo la escena activa está marcada como final e informar el desenlace al usuario.	Al llegar a una escena final (como E4, E5 o E6 del relato de referencia), el sistema imprime el texto de la escena y un mensaje que indica el fin del recorrido, y no solicita ninguna elección.
RF5.2	**Reinicio de sesión limpia.** Tras alcanzar un final o ser bloqueado, el sistema debe permitir iniciar una nueva partida restaurando por completo el estado original de la historia.	Al solicitar un reinicio, el sistema regresa a la escena de inicio y el mapa de variables pierde todos los cambios generados durante la partida anterior.
RF5.3	**Detección de bloqueo.** Si durante una partida la escena activa no es final y ninguna de sus opciones está habilitada, el sistema debe informar el bloqueo y permitir reiniciar.	Dada una historia de prueba con una escena no final cuyas opciones no se cumplen con el estado actual, el sistema muestra un mensaje de bloqueo sin cerrarse y ofrece reiniciar la partida.
N06. El escritor necesita saber por qué un archivo no es válido antes de que alguien lo juegue.		
RF6.1	**Validación previa de estructura y datos.** Antes de abrir la sesión jugable, el sistema debe escanear el archivo y rechazarlo si presenta errores críticos (ilegible, falta de campos, inicio inexistente, IDs repetidos, destinos fantasma o variables no declaradas/no booleanas, cantidad de opciones incompatible con el carácter final).	Si se intenta cargar un archivo con un destino «E99» que no existe en las escenas, la partida no inicia.

ID	Requisito	Criterio de aceptación
RF6.2	**Diagnóstico de validación.** Al rechazar un archivo, el sistema debe mostrar un mensaje claro indicando la causa exacta y, si es posible, el campo o ID que provocó el error.	Frente al error del RF6.1, el sistema muestra por consola un mensaje que indica la causa y el ID de la escena u opción involucrada, por ejemplo: «*Escena 'E2', opción 'investigar': destino 'E99' no existe*».

2.2. Requerimientos no funcionales

ID	Requisito medible	Cómo verificarlo
RNF01 — Modificabilidad	Una segunda historia con al menos tres escenas, una condición y un efecto debe ejecutarse sin modificar los archivos del motor.	Cargar la segunda historia y registrar su recorrido.
RNF02 — Claridad de errores	Cada categoría de error de contenido debe producir un diagnóstico identificable, sin iniciar una partida inválida.	Ejecutar los casos de contenido incorrecto y guardar los mensajes reales.
RNF03 — Confiabilidad	Las seis pruebas iniciales exigidas y la prueba de integración deben finalizar correctamente.	Ejecutar la suite completa y guardar el resultado.

2.3. Preguntas y supuestos

Durante la ventana del parcial no hay respuesta del cliente, por lo que cada pregunta se acompaña de un supuesto compatible con los requisitos explícitos.

ID	Pregunta de aclaración	Supuesto adoptado	Afecta
P1	La escritora quiere permitir retroceder siempre y el diseñador quiere decisiones irreversibles. ¿Debe el lector poder retroceder a una escena anterior: siempre, nunca o solo en las decisiones que el escritor marque?	Acuerdo provisional: la versión inicial no implementa retroceso. Dentro de una partida las decisiones no se deshacen y la única forma de volver al inicio es reiniciar. No se presupone la decisión posterior del cliente.	RF3.1, RF5.2
P2	¿Los valores iniciales del mapa «variables» también deben ser booleanos?	Sí. El contrato define «variables» como un mapa de booleanos iniciales, por lo que un valor inicial no booleano se rechaza como error de contenido, igual que un valor no booleano en condiciones o efectos.	RF1.2, RF6.1
P3	¿En qué orden se muestran las opciones habilitadas y cómo las elige el lector?	Se muestran en el orden en que aparecen en el archivo y el lector elige por el número de la lista. El motor identifica cada opción por su ID, de modo que el rechazo de una opción no habilitada no depende de la interfaz.	RF2.2, RF3.2

2.4. Defecto detectado en la revisión de requisitos

- **Requisito afectado:** RF5.1 (detección de finalización), en su versión previa.
- **Defecto:** una sola condición cubría dos situaciones distintas: escena «marcada como final o con cero opciones habilitadas». Una escena final y una escena no final bloqueada, es decir, con opciones pero ninguna habilitada, tienen causas y respuestas diferentes. El criterio de aceptación

las mezclaba en un solo mensaje («fin del recorrido o un bloqueo») y el bloqueo no tenía requisito ni criterio propio.

- **Riesgo:** que una escena bloqueada se tratara como un final válido y el error del relato quedara oculto, sin una prueba que lo detectara.
- **Corrección:** RF5.1 queda limitado a las escenas finales y se agrega RF5.3 para el bloqueo, con su propio criterio de aceptación. RF5.2 (reinicio) conserva su identificador.
- **Artefactos afectados:** el modelo de estados distingue *Finalizada* de *Bloqueada*, y las pruebas incluyen un caso de bloqueo.
- **Cómo se detectó:** en la revisión de los requisitos, con apoyo de una herramienta de IA.
- **Otra inconsistencia corregida:** RF3.2 decía «ignorar» la entrada inválida, pero su criterio exigía mostrar un mensaje de error. Se unificó en «rechazar».